MySQL Assignment – Real-Time Trigger Implementation

Real-time Scenarios

Scenario: You are a database developer working for an online shopping company. Your responsibility is to automate business rules using MySQL triggers.

Part 1 – Database creation

Database name: ecommerce_trigger_db

1. Create Customers table

```
CREATE DATABASE ecommerce_trigger_db;

USE ecommerce_trigger_db;


CREATE TABLE customers (
   customer_id INT PRIMARY KEY,
   customer_name VARCHAR(50),
   email VARCHAR(100),
   mobile VARCHAR(15),
   city VARCHAR(50),
   wallet_balance DECIMAL(10,2)
);
```

2. Create Products table

```
CREATE TABLE products (
   product_id INT PRIMARY KEY,
   product_name VARCHAR(100),
   category VARCHAR(50),
   price DECIMAL(10,2),
   stock_quantity INT,
   status VARCHAR(20)
);
```

3. Create Orders table

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    order_date DATE,
    total_amount DECIMAL(10,2),
    order_status VARCHAR(30),

    FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id)
);
```

4. Create Order_Items table

```
CREATE TABLE order_items (
    item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    unit_price DECIMAL(10,2),
    subtotal DECIMAL(10,2),

    FOREIGN KEY (order_id)
    REFERENCES orders(order_id),

    FOREIGN KEY (product_id)
    REFERENCES products(product_id)
);
```

5. Create Payments table

```
CREATE TABLE payments (
    payment_id INT PRIMARY KEY,
```

```
    order_id INT,
    payment_date DATE,
    payment_amount DECIMAL(10,2),
    payment_method VARCHAR(30),
    payment_status VARCHAR(30),

    FOREIGN KEY (order_id)
    REFERENCES orders(order_id)
);
```

6. Create Stock_History table

```
CREATE TABLE stock_history (
    history_id INT AUTO_INCREMENT PRIMARY KEY,
    product_id INT,
    old_stock INT,
    new_stock INT,
    change_type VARCHAR(30),
    changed_at DATETIME,

    FOREIGN KEY (product_id)
    REFERENCES products(product_id)
);
```

7. Create Order_Audit table

```
CREATE TABLE order_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT,
    old_status VARCHAR(30),
    new_status VARCHAR(30),
    changed_at DATETIME
```

);

Part 2 – Insert sample records

```
INSERT INTO customers VALUES
(1, 'Rahul', 'rahul@gmail.com', '9876543210', 'Hyderabad', 5000),
(2, 'Priya', 'priya@gmail.com', '9876543211', 'Chennai', 8000),
(3, 'Arjun', 'arjun@gmail.com', '9876543212', 'Bangalore', 3000),
(4, 'Sneha', 'sneha@gmail.com', '9876543213', 'Vizag', 10000),
(5, 'Kiran', 'kiran@gmail.com', '9876543214', 'Pune', 2000);

INSERT INTO products VALUES
(101, 'Laptop', 'Electronics', 55000, 20, 'AVAILABLE'),
(102, 'Mobile', 'Electronics', 25000, 15, 'AVAILABLE'),
(103, 'Headphones', 'Accessories', 2000, 50, 'AVAILABLE'),
(104, 'Keyboard', 'Accessories', 1500, 30, 'AVAILABLE'),
(105, 'Smart Watch', 'Electronics', 5000, 10, 'AVAILABLE'),
(106, 'Monitor', 'Electronics', 12000, 0, 'OUT_OF_STOCK'),
(107, 'Mouse', 'Accessories', 800, 40, 'AVAILABLE');

INSERT INTO orders VALUES
(1001, 1, '2026-11-01', 57000, 'PLACED'),
(1002, 2, '2026-11-02', 25000, 'CONFIRMED'),
(1003, 3, '2026-11-03', 7000, 'SHIPPED'),
(1004, 4, '2026-11-04', 12000, 'DELIVERED'),
(1005, 5, '2026-11-05', 5000, 'CANCELLED');

INSERT INTO order_items VALUES
(1, 1001, 101, 1, 55000, 55000),
```

(2, 1001, 103, 1, 2000, 2000),

(3, 1002, 102, 1, 25000, 25000),

(4, 1003, 105, 1, 5000, 5000),

(5, 1003, 107, 2, 800, 1600),

(6, 1004, 104, 8, 1500, 12000),

(7, 1005, 105, 1, 5000, 5000);

INSERT INTO payments VALUES

(501, 1001, '2026-11-01', 57000, 'UPI', 'SUCCESS'),

(502, 1002, '2026-11-02', 25000, 'CARD', 'SUCCESS'),

(503, 1003, '2026-11-03', 7000, 'NETBANKING', 'SUCCESS'),

(504, 1004, '2026-11-04', 12000, 'UPI', 'SUCCESS'),

(505, 1005, '2026-11-05', 5000, 'CARD', 'REFUNDED');

Part 3 – Trigger Assignment Questions

Section A – BEFORE INSERT Triggers

Q1. Product Price Validation

A shopping application should never allow products with a zero or negative price.

Create a BEFORE INSERT trigger on products that:

- Rejects products with a price less than or equal to zero.
- Displays the error message: Product price must be greater than zero.

Q2. Stock Validation

Whenever a new product is added:

- Stock quantity must not be negative.
- Reject insertion if stock is less than zero.
- Display an appropriate error message.

Q3. Customer Wallet Validation

A customer cannot have a negative wallet balance.

Create a BEFORE INSERT trigger on customers to validate the wallet balance.

Q4. Order Quantity Validation

Whenever a customer adds a product to an order:

- Quantity must be greater than zero.
- Reject orders with zero or negative quantities.
- Use SIGNAL SQLSTATE '45000' to generate an error.

Q5. Automatic Subtotal Calculation

Before inserting a record into order_items:

- Automatically calculate the subtotal.
- Formula: quantity * unit_price.
- Store the calculated value in the subtotal column.

Section B – AFTER INSERT Triggers

Q6. Automatic Stock Reduction

Whenever a customer purchases a product:

- Automatically reduce the corresponding product stock.
- Example: A customer purchases 3 laptops.
- Existing stock: 20.
- Remaining stock: 17.

Create an AFTER INSERT trigger on order_items.

Q7. Maintain Stock History

Whenever product stock changes because of an order:

- Store the product ID.
- Store the previous stock.
- Store the updated stock.
- Store the change type as PURCHASE.
- Store the date and time.

Use the stock_history table.

Q8. Automatically Update Order Total

Whenever a new item is added to an existing order:

- Calculate the total of all items belonging to that order.
- Automatically update orders.total_amount.

Q9. Payment Status Automation

Whenever a new payment is successfully inserted:

- Check the payment status.
- If the payment is successful, update the corresponding order status to CONFIRMED.
- If payment fails, keep the order in PLACED status.

Q10. Customer Wallet Deduction

Whenever a successful wallet payment is inserted:

- Deduct the payment amount from the customer's wallet balance.
- Reject payments if the wallet balance is insufficient.

Hint: You may add a customer_id column to the payments table or use the order relationship to identify the customer.

Section C – BEFORE UPDATE Triggers

Q11. Prevent Product Price Reduction

An e-commerce company has a rule that product prices cannot be reduced below ₹100.

Create a BEFORE UPDATE trigger on products that rejects invalid price updates.

Q12. Prevent Stock from Becoming Negative

Whenever stock is updated:

- Do not allow negative stock.
- Reject the update if the new stock quantity is less than zero.

Q13. Prevent Order Modification After Delivery

Once an order reaches DELIVERED status:

- No further status modifications should be allowed.
- Reject attempts to change the delivered order status.

Q14. Validate Payment Updates

Once a payment is marked as SUCCESS:

- Do not allow it to be changed to PENDING or FAILED.
- Reject invalid payment status transitions.

Q15. Automatically Calculate Updated Subtotal

Whenever the quantity or unit price of an order item changes:

- Automatically recalculate the subtotal.
- Update the corresponding subtotal column.

Section D – AFTER UPDATE Triggers

Q16. Maintain Order Status History

Whenever an order status changes:

- Store the order ID.
- Store the old order status.
- Store the new order status.
- Store the modification timestamp.

Use the order_audit table.

Q17. Maintain Product Stock History

Whenever an administrator manually updates product stock:

- Capture old stock.
- Capture new stock.
- Record the modification time.
- Store the change type as MANUAL_UPDATE.

Q18. Automatically Update Order Total

Whenever an existing order item's quantity changes:

- Recalculate the order total.
- Update the corresponding order record automatically.

Section E – BEFORE DELETE Triggers

Q19. Prevent Product Deletion

A company should not allow deletion of products that are currently associated with active orders.

Create a BEFORE DELETE trigger on products that:

- Checks whether the product is used in orders that are not cancelled.
- Rejects deletion if such orders exist.

Q20. Prevent Customer Deletion

A customer should not be deleted if they have any orders with status:

- PLACED
- CONFIRMED
- SHIPPED

Create a BEFORE DELETE trigger on customers.

Q21. Prevent Deleting Successful Payments

Once a payment is successfully completed:

- Do not allow its deletion.
- Allow deletion of failed or pending payment records, subject to business rules.

Section F – AFTER DELETE Triggers

Q22. Restore Product Stock

Suppose a customer removes an item from their shopping cart before order processing.

Whenever an order item is deleted:

- Restore the product quantity.
- Update the available stock.
- Maintain stock history.

Q23. Recalculate Order Total After Item Deletion

Whenever an item is deleted from an order:

- Calculate the remaining order items' total.
- Update orders.total_amount.
- If no items remain, set the order total to zero.

Q24. Maintain Deleted Order Item History

Create a new table called deleted_order_items.

Whenever an order item is deleted, automatically store:

- Item ID.
- Order ID.
- Product ID.
- Deleted quantity.
- Subtotal.
- Deletion timestamp.

Part 4 – Advanced Real-Time Business Scenarios

These questions require students to combine multiple triggers and business rules.

Q25. E-Commerce Inventory Management

Implement a complete inventory automation system.

Requirements:

1. Reduce stock when a new order item is inserted.
2. Restore stock when an order item is deleted.
3. Validate stock before accepting a purchase.
4. Maintain stock history.
5. Automatically change product status to OUT_OF_STOCK when stock reaches zero.
6. Change product status to AVAILABLE when stock becomes greater than zero.

Q26. Online Shopping Order Cancellation

When a customer cancels an order:

1. Check whether the order is already shipped or delivered.
2. Reject cancellation if it has already been shipped or delivered.
3. Restore the purchased product quantities.
4. Update the order status to CANCELLED.
5. Record the old and new statuses in the audit table.

Q27. Pharmacy Medicine Expiry Management

Create a new table called medicines with the following columns:

- medicine_id
- medicine_name

- manufacturing_date
- expiry_date
- price
- available_quantity

Business requirements:

1. Reject medicine insertion if its expiry date is earlier than the current date.
2. Reject negative medicine prices.
3. Prevent selling expired medicines.
4. Prevent stock from becoming negative.
5. Maintain medicine stock history whenever quantities change.

Q28. Hospital Appointment Management

Create two new tables:

- doctors
- appointments

Business requirements:

1. A doctor cannot have two appointments at the same date and time.
2. Reject appointments scheduled in the past.
3. Do not allow appointment deletion if the appointment is completed.
4. Maintain appointment status history.
5. Automatically update the doctor's appointment count whenever a new appointment is inserted or deleted.

Q29. Banking Transaction Management

Create two tables:

- bank_accounts
- bank_transactions

Business requirements:

1. Reject transactions with negative amounts.
2. Prevent withdrawals exceeding the account balance.

3. Automatically deduct the amount for withdrawals.
4. Automatically add the amount for deposits.
5. Maintain transaction audit history.
6. Prevent deletion of completed transactions.

Q30. Student Attendance Management

Create two tables:

- students
- attendance

Business requirements:

1. Reject attendance dates in the future.
2. Prevent duplicate attendance entries for the same student on the same date.
3. Maintain total present days for every student.
4. Update attendance statistics whenever attendance is modified.
5. Prevent deletion of attendance records after monthly attendance is finalized.

Part 5 – Trigger Management and Testing

Complete the following practical tasks:

1. Display all triggers available in the current database.
2. Display the complete definition of a particular trigger.
3. Drop an existing trigger.
4. Create multiple triggers on the same table for different events.
5. Create separate BEFORE INSERT and AFTER INSERT triggers on one table.
6. Test a trigger using multiple-row INSERT statements.
7. Verify whether an invalid operation is rejected.
8. Test how triggers behave when an SQL transaction is rolled back.
9. Verify whether audit records are maintained correctly.
10. Test the impact of foreign key constraints when deleting records.

Part 6 – Final Mini Project Challenge

Mandatory practical challenge

Project: Complete Online Shopping Database Automation

Students must implement a complete trigger-based automation system using the database created in this assignment.

Expected functionality:

- Automatic inventory management.
- Automatic order total calculation.
- Payment validation.
- Wallet balance management.
- Order cancellation management.
- Stock history tracking.
- Order audit tracking.
- Product validation.
- Prevention of invalid deletions.
- Business rule enforcement.

Final validation: Demonstrate at least 10 successful test cases and 10 failure test cases with appropriate SQL queries and outputs.

Evaluation Criteria

Evaluation	**Marks**
Database and table creation	10
Sample records and relationships	10
BEFORE triggers	15
AFTER triggers	15
Business scenario implementation	20
Advanced trigger logic	15
Testing and output demonstration	10
Explanation and presentation	5
Total	100

Trainer's instruction to students:

1. Write the SQL query for every question.
2. Create the required triggers independently.
3. Execute INSERT, UPDATE and DELETE statements to test your triggers.
4. Show both successful and unsuccessful scenarios.
5. Submit the SQL script along with screenshots of the execution results.

Important MySQL notes:

- Use NEW and OLD appropriately.
- Use SIGNAL SQLSTATE '45000' for custom validation errors.
- Use BEFORE triggers for validation and modifying incoming values.
- Use AFTER triggers for audit and related-table updates.
- Consider multi-row operations and transaction rollback.
- Avoid creating duplicate triggers that perform conflicting operations.